

Тема 16. Практика тестирования (актуальные вопросы)

- 0. Проектирование и верификация ПО**
- 1. Аспекты тестирования Web-приложений**
- 2. Аспекты тестирования Мобильных приложений**
- 3. Тестирование исключений (JUnit)**
- 4. Bug-report**

Некоторые частые ошибки

- ☐ несоответствие спецификации
- ☐ граничные условия
- ☐ off-by-one faults ($i < 10$ vs $i \leq 10$ в цикле)
- ☐ специальные случаи
- ☐ необычные ситуации
- ☐ ошибки памяти

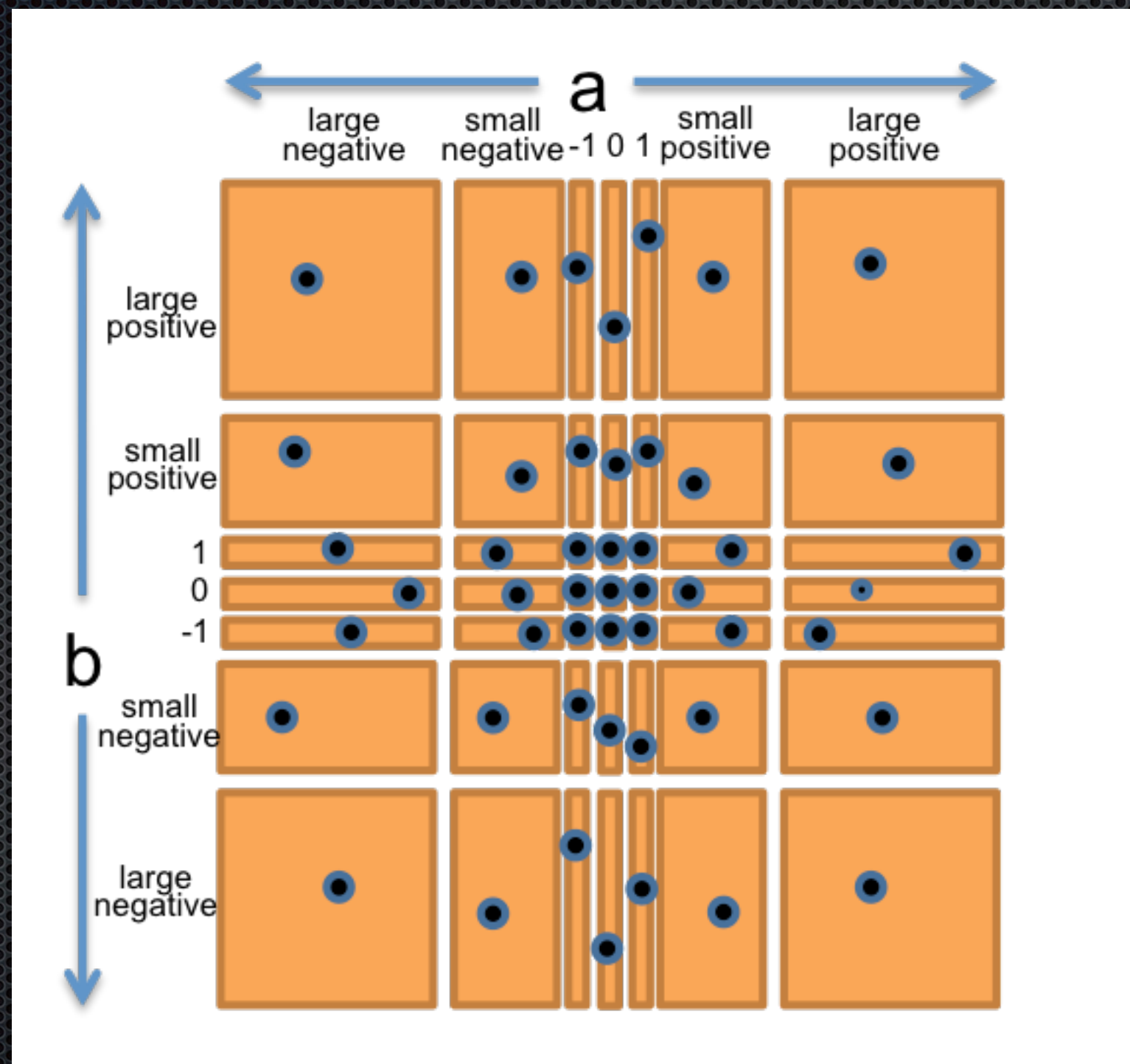
Некоторые частые ошибки

- ☐ ошибки копирования
- ☐ операторы (& vs &&)
- ☐ равенство и идентичность
- ☐ порядок параметров (bar(2, 42) vs bar(42, 2))
- ☐ бесконечные циклы, рекурсии
- ☐ недоступный код

Некоторые частые ошибки

- ☐ ошибки в расчетах (разделители)
- ☐ инициализация
- ☐ соответствие state-chart
- ☐ теория структур данных
- ☐ старые ошибки

Верификация



Типичные кейсы: Чёрный ящик

- Коллекция: Пустая / Синглтон / Много элементов
- Число: Большое / Малое / Типичное

Типичные кейсы: Белый ящик

Цикл: 0 раз, 1 раз, много раз

Ветвление: все пути

If: 2 варианта

For: 0, 1, много

While: 0, 1, много + все выходы

Рекурсия: нет вызовов, 1 вызов, много вызовов

Побочные эффекты и тестирование

```
class Cafe {
```

The `class` keyword introduces a class, much like in Java. Its body is contained in curly braces, `{` and `}`.

```
    def buyCoffee(cc: CreditCard): Coffee = {
```

A method of a class is introduced by the `def` keyword.

`cc: CreditCard` defines a parameter named `cc` of type `CreditCard`. The `Coffee` return type of the `buyCoffee` method is given after the parameter list, and the method body consists of a block within curly braces after an `=` sign.

Side effect.
Actually charges the credit card.

```
        val cup = new Coffee()
```

```
        cc.charge(cup.price)
```

No semicolons are necessary. Newlines delimit statements in a block.

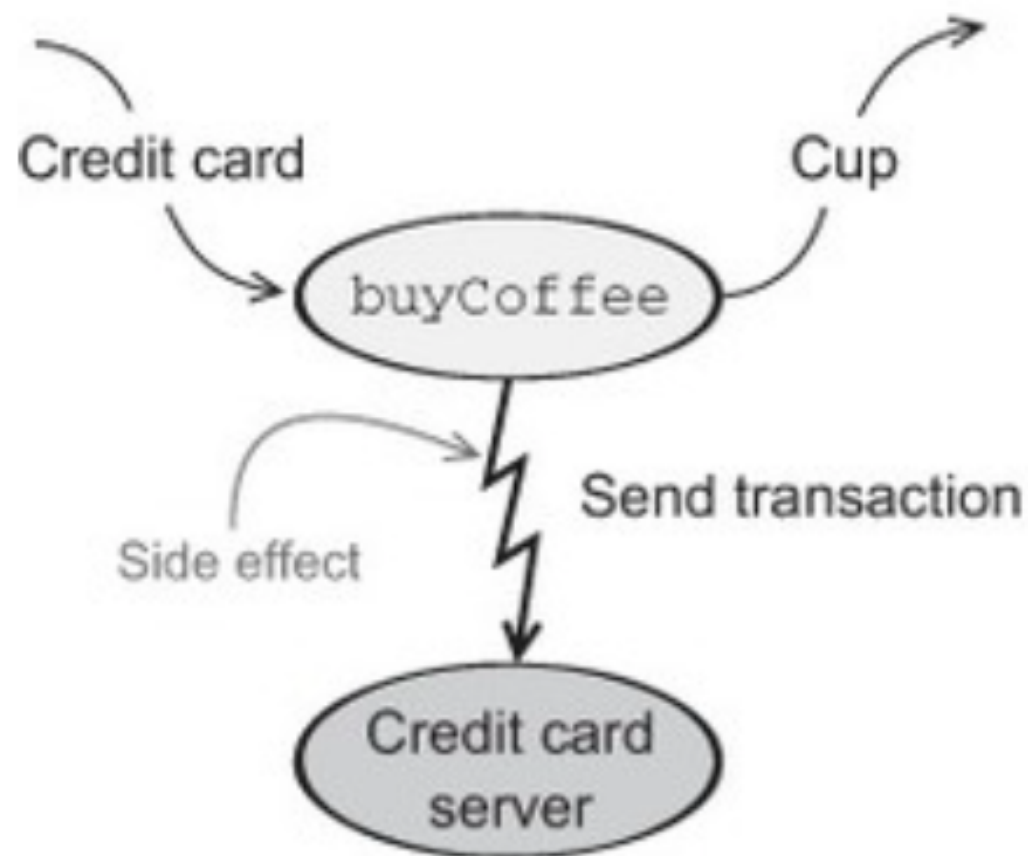
```
        cup
```

We don't need to say `return`. Since `cup` is the last statement in the block, it is automatically returned.

```
    }  
}
```

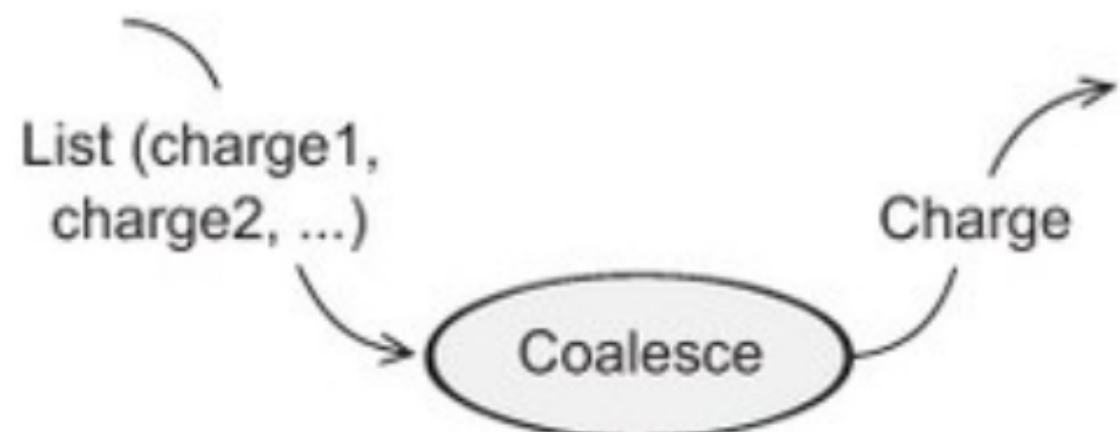

A call to buyCoffee

With a side effect



Can't test `buyCoffee` without credit card server.
Can't combine two transactions into one.

Without a side effect



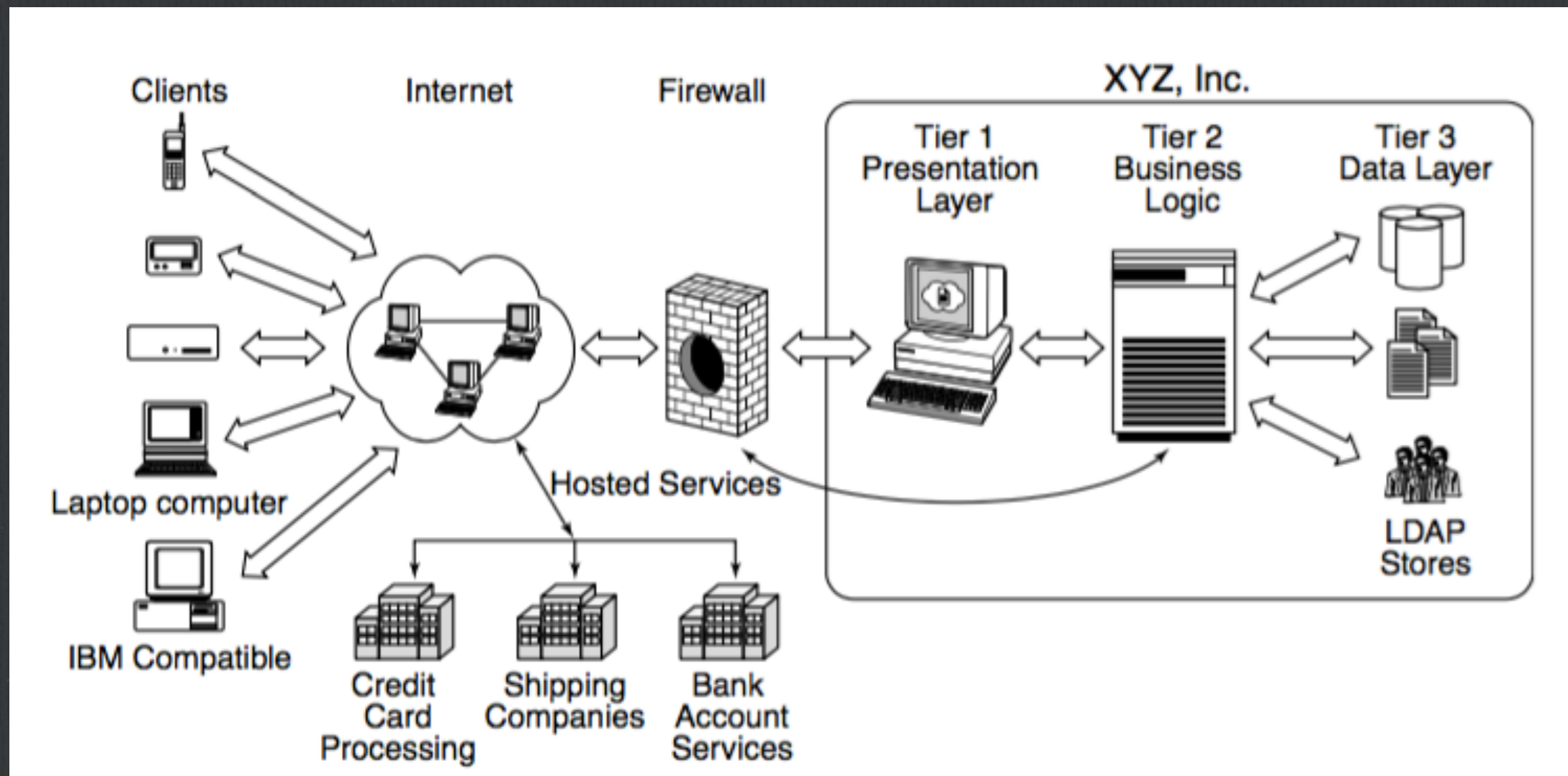
Удаление побочных эффектов

```
class Cafe {  
  def buyCoffee(cc: CreditCard): (Coffee, Charge) = {  
    val cup = new Coffee()  
    (cup, Charge(cc, cup.price))  
  }  
}
```

To create a pair, we put the `cup` and `Charge` in parentheses separated by a comma.

`buyCoffee` now returns a pair of a `Coffee` and a `Charge`, indicated with the type `(Coffee, Charge)`. Whatever system processes payments is not involved at all here.

Тестирование Web-приложений: Обобщённая детализированная архитектура



1 уровень: Web Server

- ☐ **уровень представления (presentation tier)**
- ☐ **look & feel приложения**

2 уровень: Бизнес логика

- ☐ обработка транзакций
- ☐ аутентификация пользователей
- ☐ валидация данных
- ☐ ведение логов

3 уровень: Данные

- ☐ хранение данных
- ☐ получение данных
- ☐ инфраструктура БД

Основные сложности

- ☐ большая и разнообразная пользовательская база
- ☐ бизнес-окружение
- ☐ локали пользователей
- ☐ безопасность
- ☐ компьютерная сеть

Аспекты тестирования уровня представления

- ☐ тестирование контента
- ☐ тестирование структуры сайта
- ☐ тестирование окружения пользователя

Аспекты тестирования бизнес слоя

- ☐ производительность
- ☐ валидность данных
- ☐ транзакции

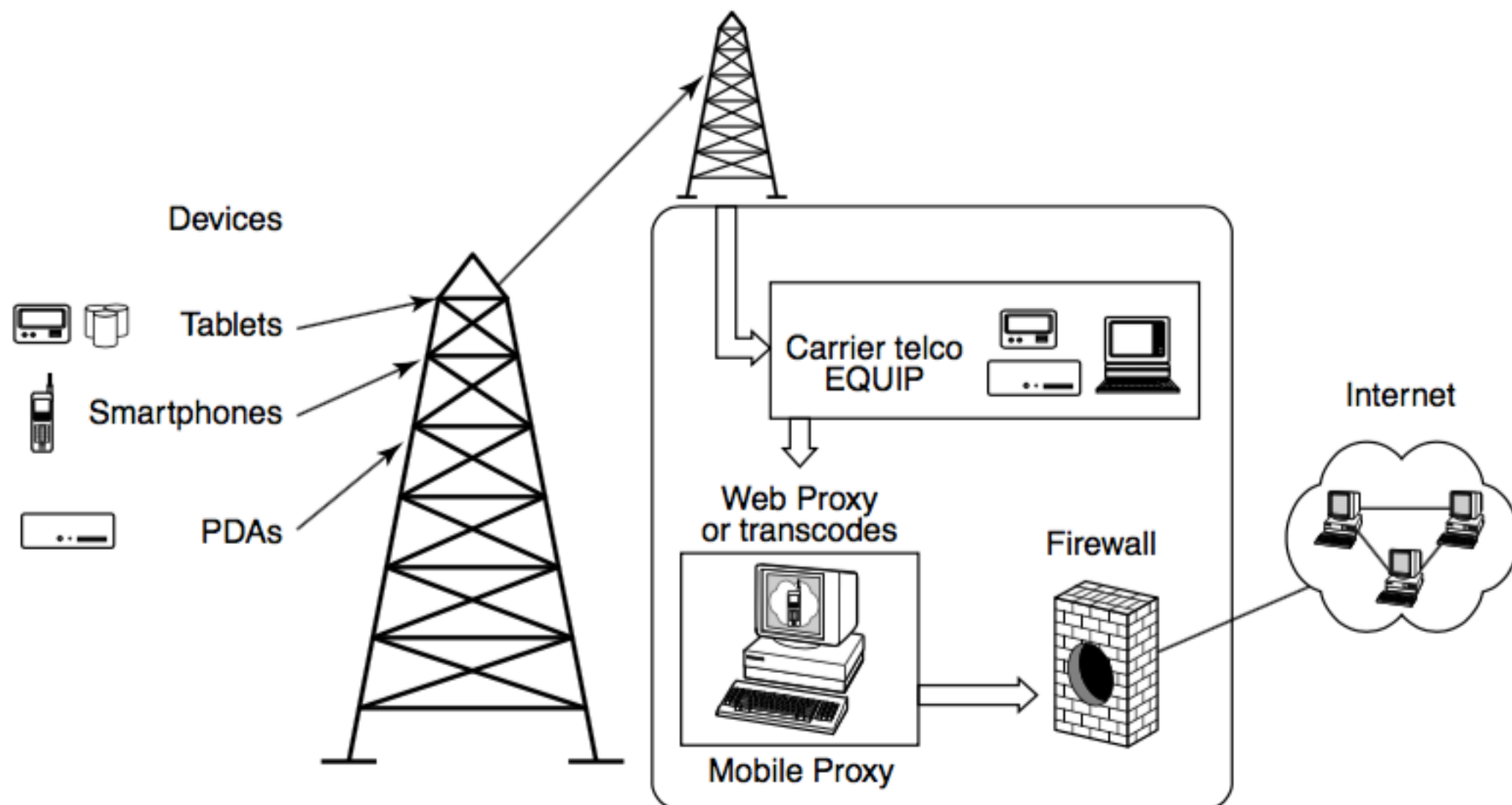
Аспекты тестирования уровня данных

- ☐ **время отклика**
- ☐ **целостность данных**
- ☐ **устойчивость к ошибкам и восстанавливаемость**

Примеры аспектов тестирования

Presentation Tier	Business Tier	Data Tier
Ensure fonts are the same across browsers.	Verify proper calculation of sales tax and shipping charges.	Ensure database operations meet performance goals.
Confirm that all links point to valid files or websites.	Ensure documented performance rates are met for response times and throughput rates.	Verify that data are stored correctly and accurately.
Verify that graphics are the correct resolution and size.	Verify that transactions complete properly.	Verify that you can recover using current backups.
Spell-check each page.	Confirm that failed transactions roll back correctly.	Test failover or redundancy operations.
Have a copy editor check grammar and style.	Ensure data are collected correctly.	Test for proper data encryption and security (credit card and user's personal information, in particular).
Check cursor positioning when page loads to ensure it is in the correct text box.		

Тестирование мобильных приложений: обобщённая архитектура



Тестирование мобильных приложений: сложности

- ☐ разнообразие устройств
- ☐ сетевая инфраструктура
- ☐ написание скриптов
- ☐ юзабилити
- ☐ реальное устройство / эмулятор

Тестирование мобильных приложений: некоторые аспекты

- ☐ установка / удаление приложения
- ☐ выход / вход в приложение
- ☐ сеть
- ☐ обработка звонков / сообщений
- ☐ низкая память
- ☐ обратная связь
- ☐ низкий уровень заряда / заряд батареи
- ☐ способы ввода информации и маппинг

Тестирование исключений: аннотации

```
@Test(expected = IndexOutOfBoundsException.class)
public void empty() {
    new ArrayList<Object>().get(0);
}
```


Тестирование исключений: try/catch

```
@Test
public void testExceptionMessage() {
    try {
        new ArrayList<Object>().get(0);
        fail("Expected an IndexOutOfBoundsException to be thrown");
    } catch (IndexOutOfBoundsException anIndexOutOfBoundsException) {
        assertThat(anIndexOutOfBoundsException.getMessage(), is("Index: 0, Size: 0"));
    }
}
```


Тестирование исключений: аннотация @Rule

```
@Rule
public ExpectedException thrown = ExpectedException.none();

@Test
public void shouldTestExceptionMessage() throws IndexOutOfBoundsException {
    List<Object> list = new ArrayList<Object>();

    thrown.expect(IndexOutOfBoundsException.class);
    thrown.expectMessage("Index: 0, Size: 0");
    list.get(0); // execution will never get past this line
}
```

```
thrown.expectMessage(JUnitMatchers.containsString("Size: 0"));
```


Тестирование исключений: assert (Java 8)

```
@Test
public void verifiesTypeAndMessage() {
    assertThrown(new DummyService()::someMethod)
        .assertInstanceOf(RuntimeException.class)
        .hasMessage("Runtime exception occurred")
        .hasMessageStartingWith("Runtime")
        .hasMessageEndingWith("occurred")
        .hasMessageContaining("exception")
        .hasNoCause();
}
```


Тестирование исключений: аннотация BDD (Catch Exception)

```
@Test
public void testFooThrowsIndexOutOfBoundsException() {

    when(foo).doStuff();

    then(caughtException()).isInstanceOf(IndexOutOfBoundsException.class);

}
```


Bug Report

- ☐ версия продукта
- ☐ окружение
- ☐ описание
- ☐ приоритетность
- ☐ шаги
- ☐ наблюдаемый результат
- ☐ ожидаемое поведение
- ☐ дальнейшие действия